Migration Report: Repository Layer

Current state

ExpenseMate is a Python command-line app that uses SQLite. DatabaseManager still handles the connection, creates the tables, runs database operations, saves changes, and offers old-style methods for transactions and budgets. Data lives in two tables:

- transactions: id, amount, category, trans_type, date, description (indexes on date and category).
- budgets: id, category, monthly_limit, month_year (unique on category + month_year).

The move is partly done. TransactionRepository now owns the transaction SQL and turns rows into Transaction objects. BudgetRepository owns the budget SQL, including insert-or-update for a category and month. Both validate before saving and use parameterized SQL.

ExpenseService, AnalyticsEngine, and DataPortabilityManager still receive a DatabaseManager and call its original methods, which now pass the work to the repositories. The database boundary exists, but services are not rewired yet.

Target state

Final flow: CLI/controller -> services -> repositories -> DatabaseManager -> SQLite.

- DatabaseManager only manages the connection and sets up the schema.
- The two repositories are the only places with transaction and budget SQL.
- Services, analytics, and CSV import/export use repository methods (add, get_all, get_by_month, save, get, get_for_month). The old wrappers may stay for now, but new code must not use them.
- Nothing else changes: schema, stored values, date formats, validation, CLI behavior, currency conversion, budget thresholds, analytics, CSV format, and backups.

Reason for migration

DatabaseManager used to do everything: connections, schema, SQL, validation, row mapping, and persistence for the whole app. That tied the whole app to one database class and made storage hard to test alone. Repositories give each data type one owner, allow separate tests, cut repeated SQL, and let storage change later without rewriting business logic.

Migration steps

1. Keep the existing SQLite file. CREATE TABLE IF NOT EXISTS means no records are deleted or copied.
2. Keep DatabaseManager.get_connection() as the one connection source, with its sqlite3.Row mapping.
3. Use TransactionRepository for inserts, full reads, and month reads, keeping the newest-first date/ID order.
4. Use BudgetRepository for saving, looking up, and listing budgets by month, keeping the unique upsert and category order.
5. Keep the DatabaseManager wrappers during the transition so existing services, CLI, imports, and exports keep working.
6. Change the service, analytics, and portability constructors to take repositories, then update the ExpenseMateCLI wiring. Remove wrapper use only after regression tests pass.
7. Record the cutover and test result in the changelog. No schema migration is needed.

Risks, compatibility, and rollback

- Main risk: behavior changing by accident while rewiring. Month filters must keep using ISO YYYY-MM / YYYY-MM-DD, budget updates must not create duplicates, and CSV import must keep per-row error reporting. SQL errors must still surface as ValueError or RuntimeError.
- Compatible while wrappers remain. Before rewiring, copy the .db file and check transaction and budget counts and sample values.
- Rollback: restore the previous code and point it at the unchanged database copy. No rollback SQL is needed because no data is transformed. Never delete the original database or test on the live file.

Verification and acceptance criteria

Run from the repository root: python -m unittest test_repositories.py test_expense_mate.py

- test_repositories.py: transaction insert and month retrieval, and budget upsert with no duplicate rows.
- test_expense_mate.py: validation, reads, summaries, budget alerts, currency conversion, CSV round-trip, analytics.
- After the final cutover, rerun both and add tests showing services use repositories and run no SQL.

Accepted when: records stay readable, table and index definitions are unchanged, all tests pass, no SQL remains outside the repositories and DatabaseManager, and CLI workflows give the same results before and after.